

智能巡检车 PRO 版实验手册

雷达避障实战

目录

1 实验背景	2
2 实验原理	2
3 实验目的	2
4 实验环境	3
5 实验流程	3
6 实验任务	5
1.1 任务一 为 iCar 智能小车上电，启动系统	5
1.2 任务二 大禹 200 上电，启动控制 APP	6
1.3 任务三 使用控制 APP 控制 iCar 智能小车的运动	9
1.4 任务四 启动 iCar 智能小车系统中的 docker 开发环境	11
1.5 任务五 在 docker 镜像环境中运行预置的 ROS2 雷达避障功能	17
1.6 任务六 修改 ROS2 雷达避障功能代码	22
1.7 任务七 编写 python 代码控制底盘运动	30
7 实验结果	36
8 实验资源释放	36
9 实验小结	37

1

实验背景

iCar 智能小车提供了一个跨物联网、自动化、机器人、人工智能、自动驾驶、智能语音等多学科的综合研发平台，运行了通用机器人 ROS2 系统，以及基于鸿蒙 AT32 控制板的底盘麦轮运动控制系统，搭载了深度相机、激光雷达等多种传感器，可以进行多学科及跨学科的开发及实验研究，其中结合激光雷达的测距功能，可以快速开发出雷达避障功能。

2

实验原理

1. iCar 智能小车的启动及控制，涉及 APP 和上位机的网络通信及上位机和底盘的串口通信。
2. Docker 的使用，可以将 ROS 运行环境固化在 Docker 中，方便复用以及更新。
3. ROS2 开发环境，方便各种传感器的整合及数据互通。
4. 激光雷达的扫描和测距功能的了解和实际使用。
5. 通过 ROS2 应用调用串口接口实现对小车底盘的运动控制。
6. 通过 ROS2 的话题订阅和发布，进行数据互通，获得雷达扫描数据。
7. 根据雷达测距扫描数据，进行避障场景的代码逻辑处理。

3

实验目的

1. 熟练掌握 iCar 智能小车的运动控制和使用。
2. 熟练掌握 Docker 开发环境的使用和常用指令。

3. 熟练掌握 Ros2 开发环境的常见概念和常用指令。
4. 掌握使用 Ros2 平台对雷达数据及底盘速度话题的订阅和发布。
5. 掌握结合雷达数据来控制底盘麦轮运动的方法,实现 iCar 智能小车运动过程中的实时避障。

4 实验环境

代码开发工具版本: Visual Studio Code 1.93.0

1. 测试工具: 大禹 200 开发板硬件
2. 操作系统版本: Ubuntu18.04/Ubuntu20.04
3. 机器人操作系统 ROS 版本: ROS2
4. Docker 版本: 24.0.0 及以上
5. 开发语言: Python 3.8 及以上 , C++ 11 及以上

5 实验流程

实验整体流程图如下所示:



iCar 智能小车上电后，机器人系统启动，服务程序自动启动，同时给大禹 200 开发板上电，让大禹 200 系统启动，使用大禹 200 上安装的 APP 来操控机器人的运动、转向及巡线等。

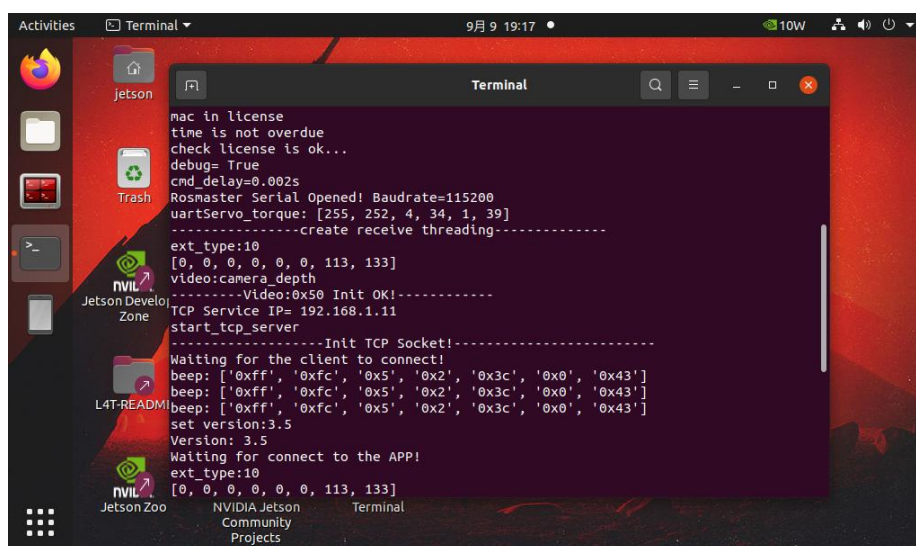
在 iCar 智能小车系统中，可以编写控制小车的代码、编写获取雷达数据进行避障的代码，并在 iCar 智能小车系统中的 Docker 镜像中的 ROS2 环境中部署运行，观察 iCar 智能小车的雷达避障行为。

6

实验任务

6.1 任务一 为 iCar 智能小车上电，启动系统

按下 iCar 智能小车身上的电源按钮，为 iCar 智能小车上电后，iCar 智能小车顶部的屏幕被点亮，可以看到 iCar 智能小车系统启动过程的屏幕输出，启动成功后，进入 Ubuntu20.04 系统，并自动启动上位机服务程序：



```
mac in license
time is not overdue
check license is ok...
debug= True
cmd_delay=0.002s
Rosmaster Serial Opened! Baudrate=115200
uartServo_torque: [255, 252, 4, 34, 1, 39]
-----create receive threading-----
ext_type:10
[0, 0, 0, 0, 0, 0, 113, 133]
video:camera_depth
-----Video:0x50 Init OK!-----
TCP Service IP= 192.168.1.11
start_tcp_server
-----Init TCP Socket!-----
Waiting for the client to connect!
beep: ['0xff', '0xfc', '0x5', '0x2', '0x3c', '0x0', '0x43']
beep: ['0xff', '0xfc', '0x5', '0x2', '0x3c', '0x0', '0x43']
beep: ['0xff', '0xfc', '0x5', '0x2', '0x3c', '0x0', '0x43']
set version:3.5
Version: 3.5
Waiting for connect to the APP!
ext_type:10
[0, 0, 0, 0, 0, 0, 113, 133]
```

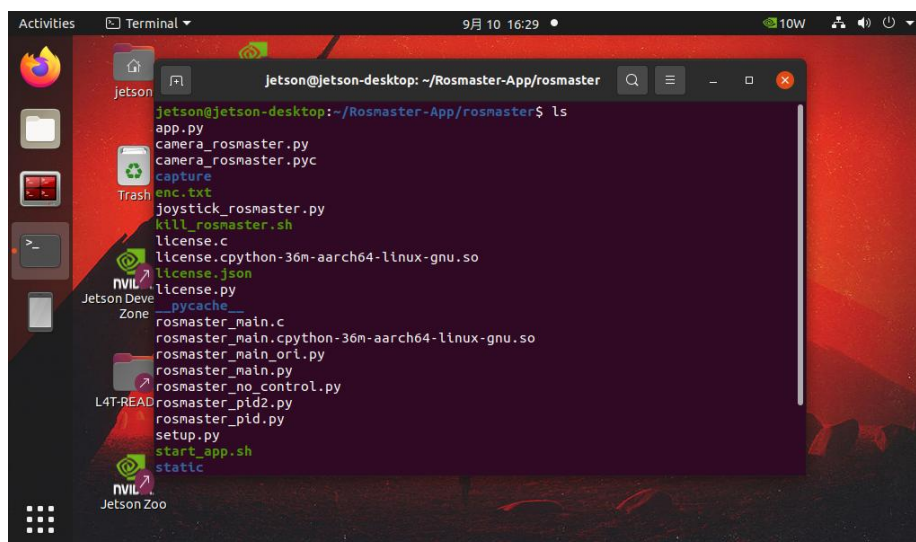
在启动上位机服务程序过程中，会听到车上蜂鸣器的哔、哔、哔三声响，说明程序启动正常。

记住此程序对外的服务 IP 地址，例如上图中的 192.168.1.11，这个地址在下一步的 APP 启动时要用到。

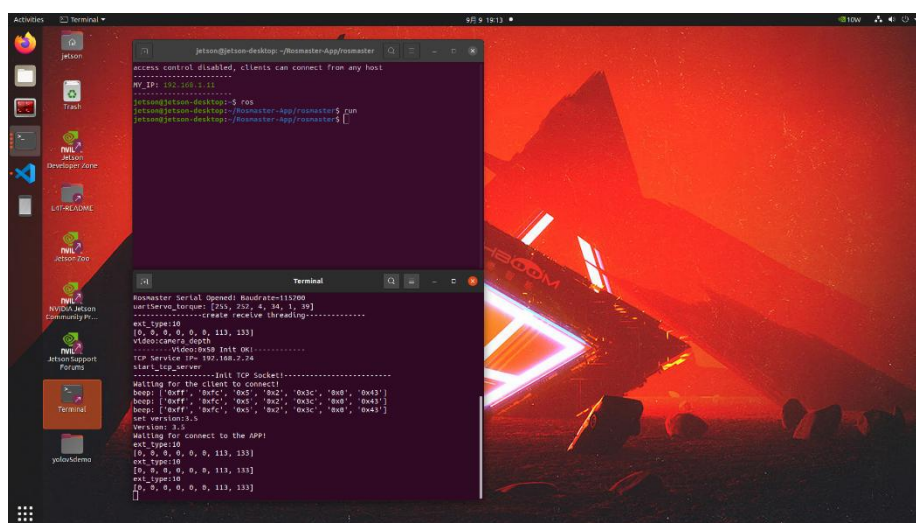
也可以手动启动该程序，操作步骤如下：

1. 通过 iCar 智能小车的外置串口，连接键盘鼠标。
2. 先使用鼠标关闭现有运行窗口，则程序退出运行。

3. 使用鼠标点击左边栏上的 Terminal 图标，启动新窗口
4. 在新窗口中输入指令 `ros`，则会进入上位机程序所在的目录。



5. 在新窗口中继续输入 run 指令，则会弹出一个新的 Terminal 并启动、运行上位机服务程序。



在启动程序过程中，会听到车上蜂鸣器的哔、哔、哔三声响，说明程序启动正常。

6.2 任务二 大禹 200 上电，启动控制 APP

1. 为大禹 200 终端连接电源线通电, 启动大禹 200 终端, 该终端上已安装了服务机器人控制 APP。



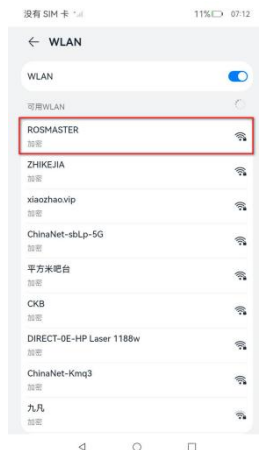
2. 在大禹 200 屏幕上，上滑解锁，并点击设置卡片，进入设置界面。



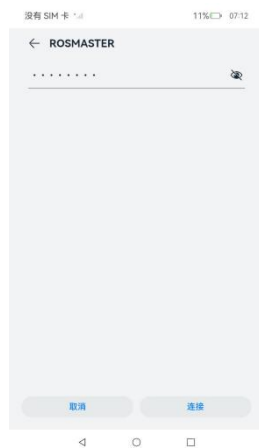
3. 在设置界面选择 WLAN。



4. 在 WLAN 界面看到 iCar 智能小车开启的无线热点 ROSMASTER，点击设置。



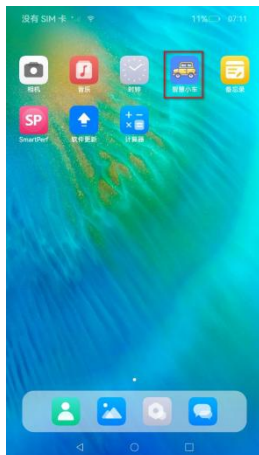
5. 输入 ROSMASTER 网络的默认接入密码：12345678，并点击“连接”按钮。



6. 看到 ROSMASTER 网络已连接,说明大禹 200 可以连上 iCar 智能小车的热点网络了。



7. 点击屏幕下方的方框，并删除设置界面，回到大禹 200 的主界面，选择启动 iCar 智能小车的控制 APP。



8. APP 以横屏展示, 确认 IP 地址和第 5 步中的上位机系统的 IP 地址是一致的, 点击“连接”按钮。



9. 进入 APP 的功能界面。



可以选择“麦科朗姆”和“单独控制”两个功能卡片。

6.3 任务三 使用控制 APP 控制 iCar 智能小车的运动

1. 在智能小车控制 APP 上, 选择“麦科朗姆”卡片, 可进入对麦科朗姆轮的运动控制界

面。

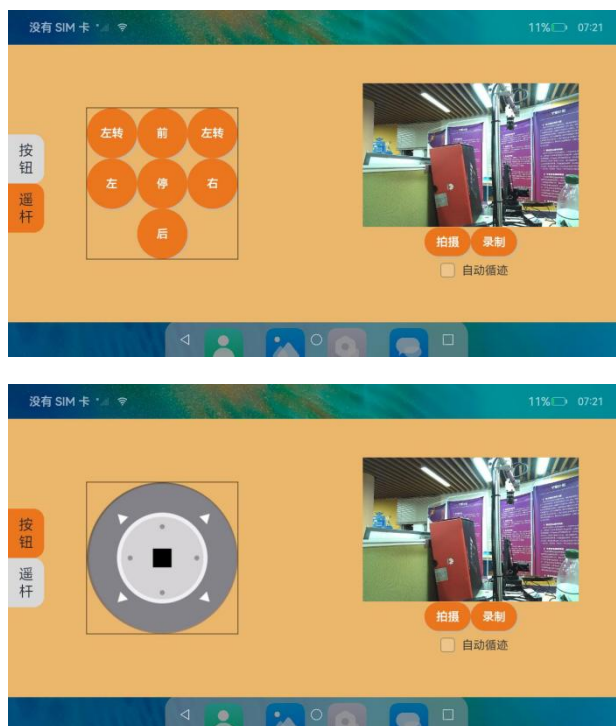


可通过摇杆控制 iCar 智能小车的底盘麦轮的整体运动，也可以设置单个轮子的转速。

注意：对于单轮设置更新速度时，为了便于观察，可以将 iCar 智能小车底部放一个支撑物，使其底盘四个轮子悬空，这样可以观测 APP 下发指令后，单轮的运动变化。

进行摇杆控制时，可以把 iCar 智能小车放到地面，可以看到 iCar 智能小车随着摇杆的方向进行前进、后退、平移等各种运动。

2. 选择“单独控制”卡片，进入控制界面，可以选择“按钮”和“摇杆”两种控制方式



选择“按钮”菜单，可使用按钮控制 iCar 智能小车的前进、后退、左平移、右平移、左转、

右转、停止等运动行为。

选择“摇杆”菜单，可使用摇杆控制 iCar 智能小车前进、后退、平移等运动行为。

APP 界面的右边部分是 iCar 智能小车身上的相机采集到的实时画面，可点击“拍摄”或者“录制”按钮，对相机画面进行取图或者视频录制，用于 AI 训练、巡检结果保存等目的。

相关数据存储在上位机系统的如下路径：~/Rosmaster-App/rosmaster/capture/，可以通过连接键盘、鼠标到 iCar 智能小车身上的 USB 接口上，到该目录下查看保存的图片或者视频，并通过 U 盘或者网络将图片或者视频拷贝出去，用于人工智能目标检测识别的数据素材。

3. 开启和关闭巡线功能

在地面布置巡线标记（白底黑线），在 iCar 智能小车的控制 APP 上勾选“自动循迹”，可触发 iCar 智能小车进行自动巡线运动。



iCar 智能小车将沿着黑线指示的路线前进或者转弯。

要停止自动巡线，在控制 APP 界面上取消勾选“自动循迹”即可。

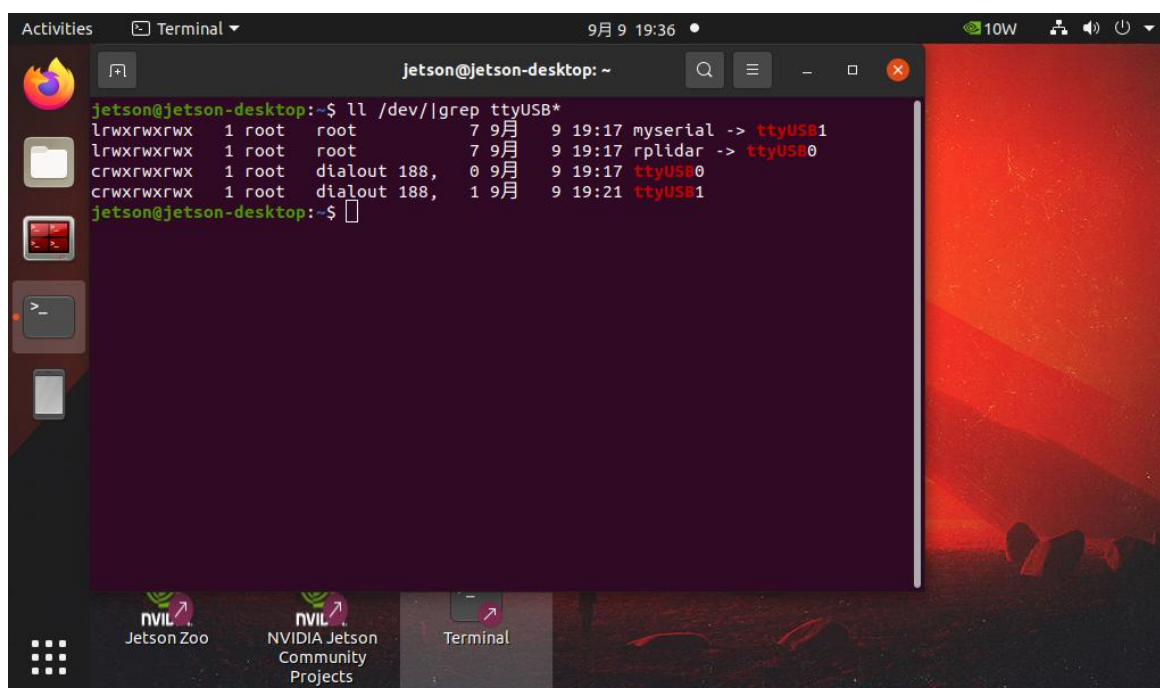
6.4 任务四 启动 iCar 智能小车系统中的 docker 开发环境

在 iCar 智能小车的系统中，我们预置了用 docker 制作的开发环境，将 ROS2 系统及所依赖的各种驱动程序、各种库等都在 docker 镜像中部署好了，当需要做开发或测试时，

只需要启动对应的 docker 镜像，将修改的代码上传到 docker 上的 ROS2 环境，完成编译并部署运行，即可看到 iCar 智能小车的各种实时运行效果，极大的简化了开发环境部署的巨大工作量。

本任务是使用 docker 指令，正确启动 ROS2 开发环境中的 docker 镜像。

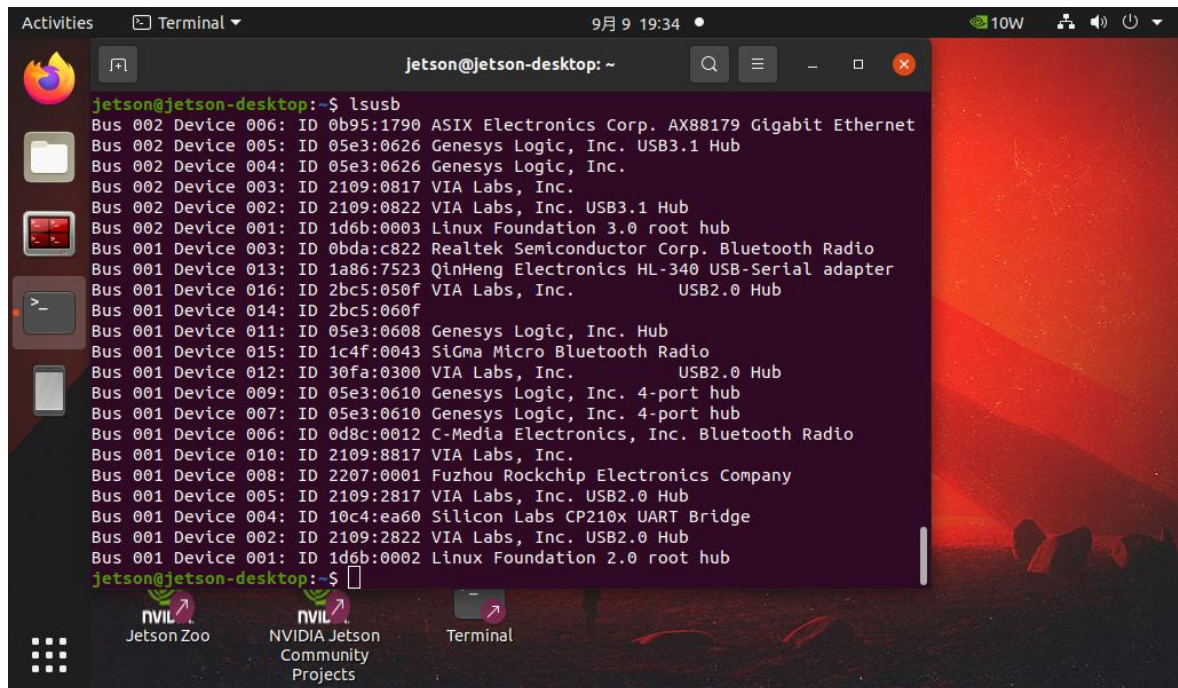
1. 确认 iCar 智能小车的雷达 USB 接线、底盘控制 USB 接线 OK。



```
jetson@jetson-desktop:~$ ll /dev | grep ttyUSB*
lrwxrwxrwx 1 root root 7 9月 9 19:17 myserial -> ttyUSB1
lrwxrwxrwx 1 root root 7 9月 9 19:17 rplidar -> ttyUSB0
crwxrwxrwx 1 root dialout 188, 0 9月 9 19:17 ttyUSB0
crwxrwxrwx 1 root dialout 188, 1 9月 9 19:21 ttyUSB1
jetson@jetson-desktop:~$
```

能看到 myserial（底盘控制接线）、rplidar（雷达接线），说明相应的 USB 接线正确，Ubuntu 系统能正常识别正确。

2. 确认深度相机的编号。

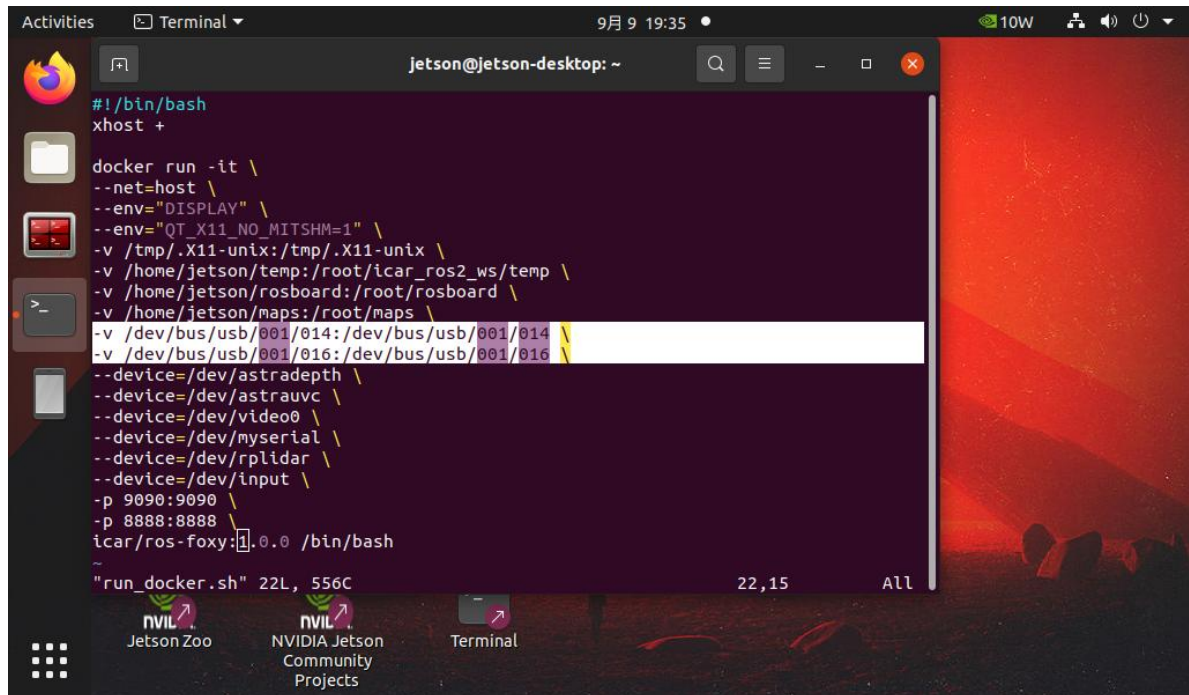


```
jetson@jetson-desktop:~$ lsusb
Bus 002 Device 006: ID 0b95:1790 ASIX Electronics Corp. AX88179 Gigabit Ethernet
Bus 002 Device 005: ID 05e3:0626 Genesys Logic, Inc. USB3.1 Hub
Bus 002 Device 004: ID 05e3:0626 Genesys Logic, Inc.
Bus 002 Device 003: ID 2109:0817 VIA Labs, Inc.
Bus 002 Device 002: ID 2109:0822 VIA Labs, Inc. USB3.1 Hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 0bda:c822 Realtek Semiconductor Corp. Bluetooth Radio
Bus 001 Device 013: ID 1a86:7523 QinHeng Electronics HL-340 USB-Serial adapter
Bus 001 Device 016: ID 2bc5:050f VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 014: ID 2bc5:060f VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 011: ID 05e3:0608 Genesys Logic, Inc. Hub
Bus 001 Device 015: ID 1c4f:0043 Sigma Micro Bluetooth Radio
Bus 001 Device 012: ID 30fa:0300 VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 009: ID 05e3:0610 Genesys Logic, Inc. 4-port hub
Bus 001 Device 007: ID 05e3:0610 Genesys Logic, Inc. 4-port hub
Bus 001 Device 006: ID 0d8c:0012 C-Media Electronics, Inc. Bluetooth Radio
Bus 001 Device 010: ID 2109:8817 VIA Labs, Inc.
Bus 001 Device 008: ID 2207:0001 Fuzhou Rockchip Electronics Company
Bus 001 Device 005: ID 2109:2817 VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 004: ID 10c4:ea60 Silicon Labs CP210x UART Bridge
Bus 001 Device 002: ID 2109:2822 VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
jetson@jetson-desktop:~$
```

观察设备编号有“2bc5”的两行，记录第四列的编号为 014、016（根据实际情况，编号有可能变化）。

如果只看到一行有“2bc5”，则需要重新拔插深度相机的 USB 连线，直到系统正确识别出现两行有“2bc5”。

3. 将上面的两个编号更新到~/run_docker.sh 文件中如下两行。

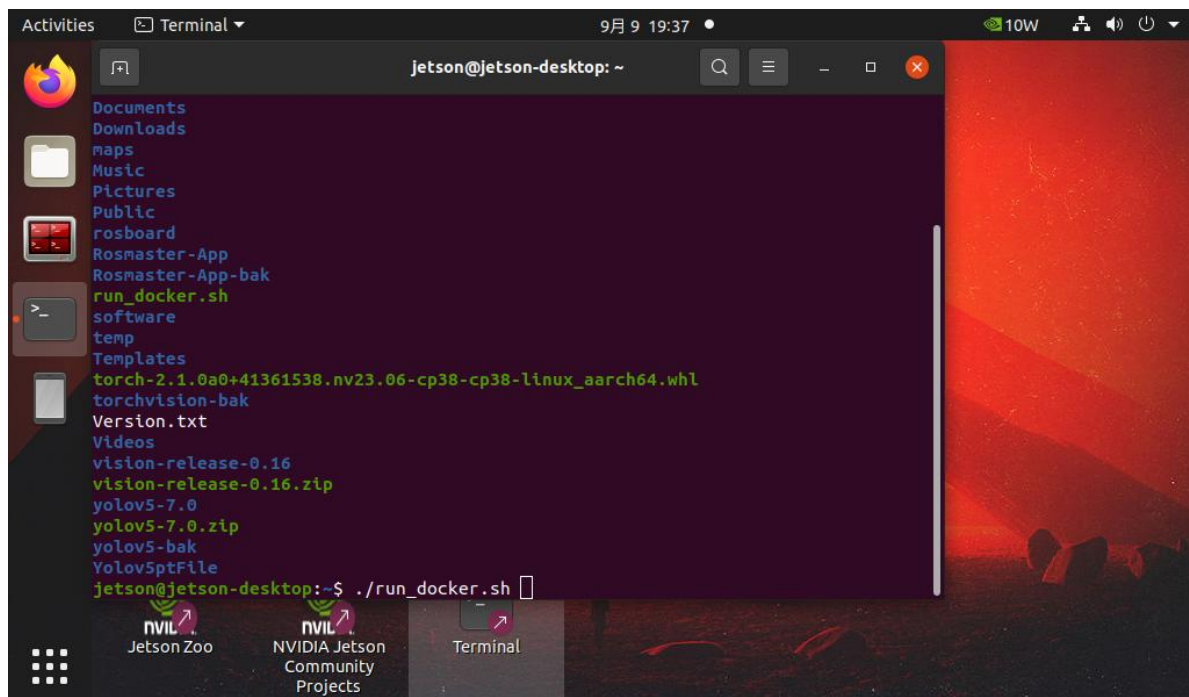


```
jetson@jetson-desktop: ~  
#!/bin/bash  
xhost +  
  
docker run -it \  
--net=host \  
--env="DISPLAY" \  
--env="QT_X11_NO_MITSHM=1" \  
-v /tmp/.X11-unix:/tmp/.X11-unix \  
-v /home/jetson/temp:/root/icar_ros2_ws/temp \  
-v /home/jetson/rosboard:/root/rosboard \  
-v /home/jetson/maps:/root/maps \  
-v /dev/bus/usb/001/014:/dev/bus/usb/001/014 \  
-v /dev/bus/usb/001/016:/dev/bus/usb/001/016 \  
--device=/dev/astradept \  
--device=/dev/astrauvc \  
--device=/dev/video0 \  
--device=/dev/myserial \  
--device=/dev/rplidar \  
--device=/dev/input \  
-p 9090:9090 \  
-p 8888:8888 \  
icar/ros-foxy:1.0.0 /bin/bash  
"run_docker.sh" 22L, 556C 22,15 All
```

保存退出。

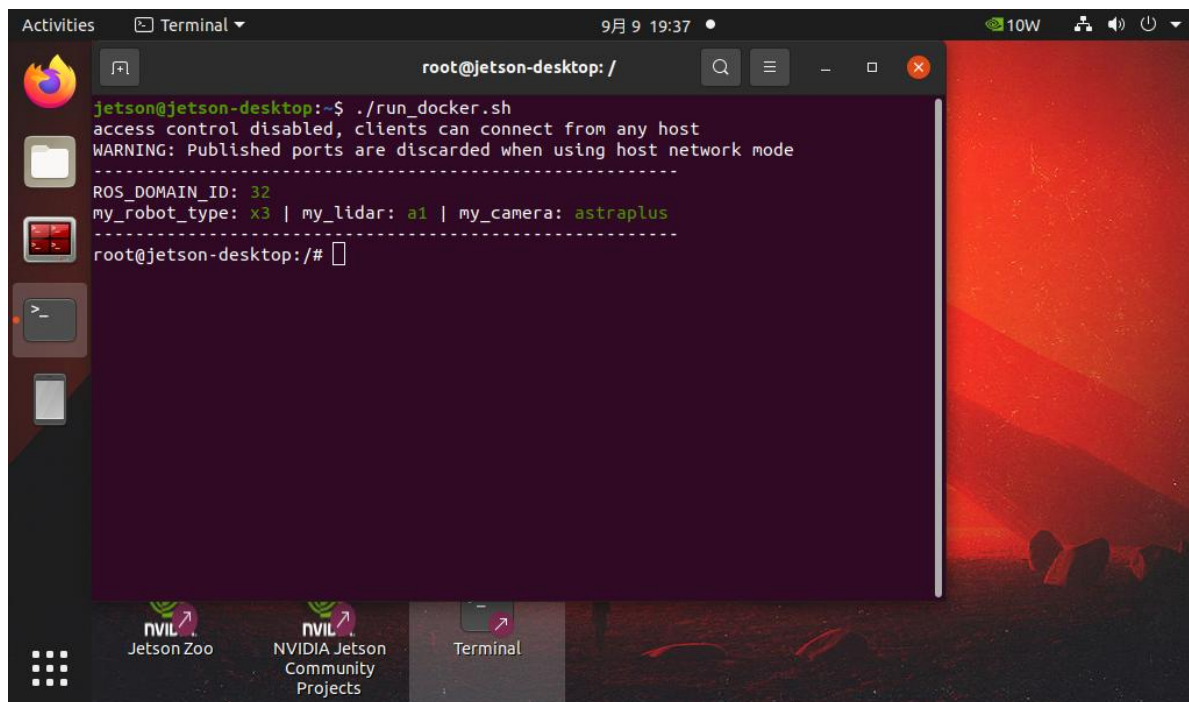
输入指令 cd。

4. 输入指令./run_docker.sh，启动 docker 镜像。

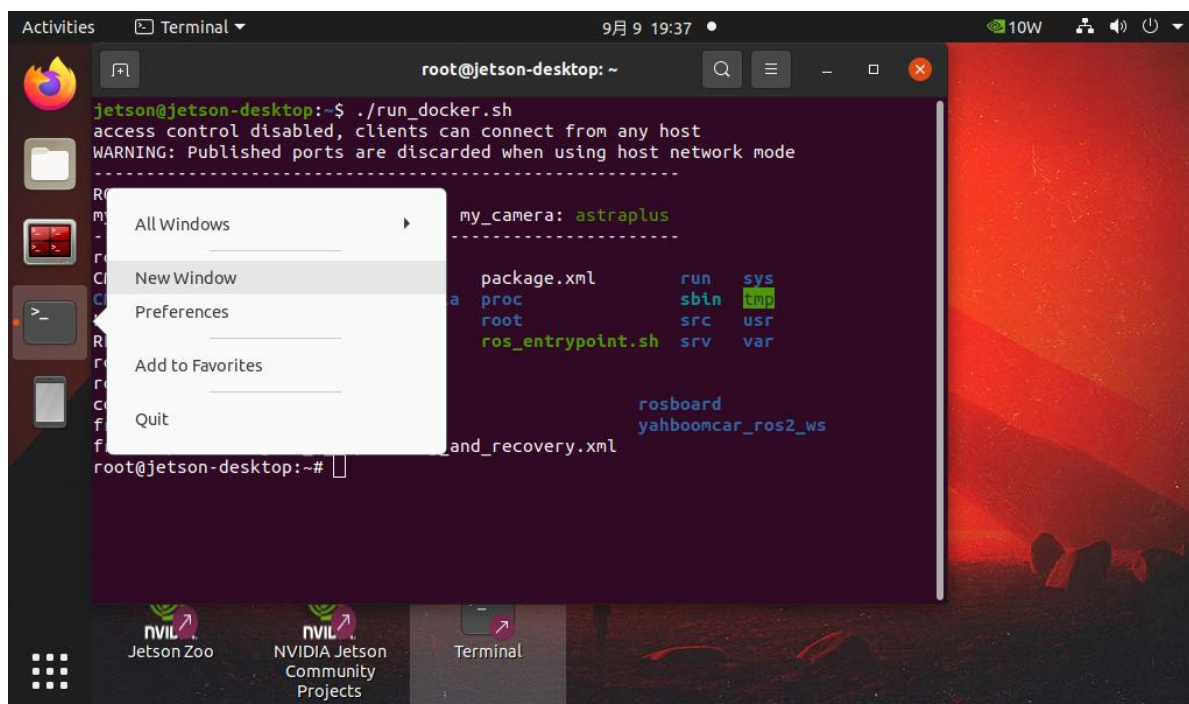


```
Documents  
Downloads  
maps  
Music  
Pictures  
Public  
rosboard  
Rosmaster-App  
Rosmaster-App-bak  
run_docker.sh  
software  
temp  
Templates  
torch-2.1.0a0+41361538.nv23.06-cp38-cp38-linux_aarch64.whl  
torchvision-bak  
Version.txt  
Videos  
vision-release-0.16  
vision-release-0.16.zip  
yolov5-7.0  
yolov5-7.0.zip  
yolov5-bak  
Yolov5ptFile  
jetson@jetson-desktop:~$ ./run_docker.sh
```

如无异常，出现如下信息，则说明进入 docker 镜像启动成功，并已进入 docker 环境。



5. 以启动预置的雷达避障功能为例，还需要启动两个 docker 窗口备用（用于启动 ROS2 应用节点）。



```

jetson@jetson-desktop:~$ access control disabled, clients can connect from any host
access control disabled, -----
WARNING: Published ports MY_IP: 192.168.1.11
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
jetson@jetson-desktop:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
e49c1206ae0f   icar/ros-foxy:1.0.0   "/bin/bash"            34 seconds ago   Up 33 second
jetson@jetson-desktop:~$ docker exec -it e49c1206ae0f /bin/bash
root@jetson-desktop:/#

```

使用 docker ps 命令获得容器的 ID。

```

jetson@jetson-desktop:~$ access control disabled, clients can connect from any host
access control disabled, -----
WARNING: Published ports MY_IP: 192.168.1.11
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
jetson@jetson-desktop:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
e49c1206ae0f   icar/ros-foxy:1.0.0   "/bin/bash"            34 seconds ago   Up 33 second
jetson@jetson-desktop:~$ docker exec -it e49c1206ae0f /bin/bash
root@jetson-desktop:/#

```

使用 docker exec -it e49c /bin/bash 指令可以进入指定 id 的容器中。

上述一共启动了三个 Docker 终端，将在后面的任务中，分别启动相应的 ROS2 应用节

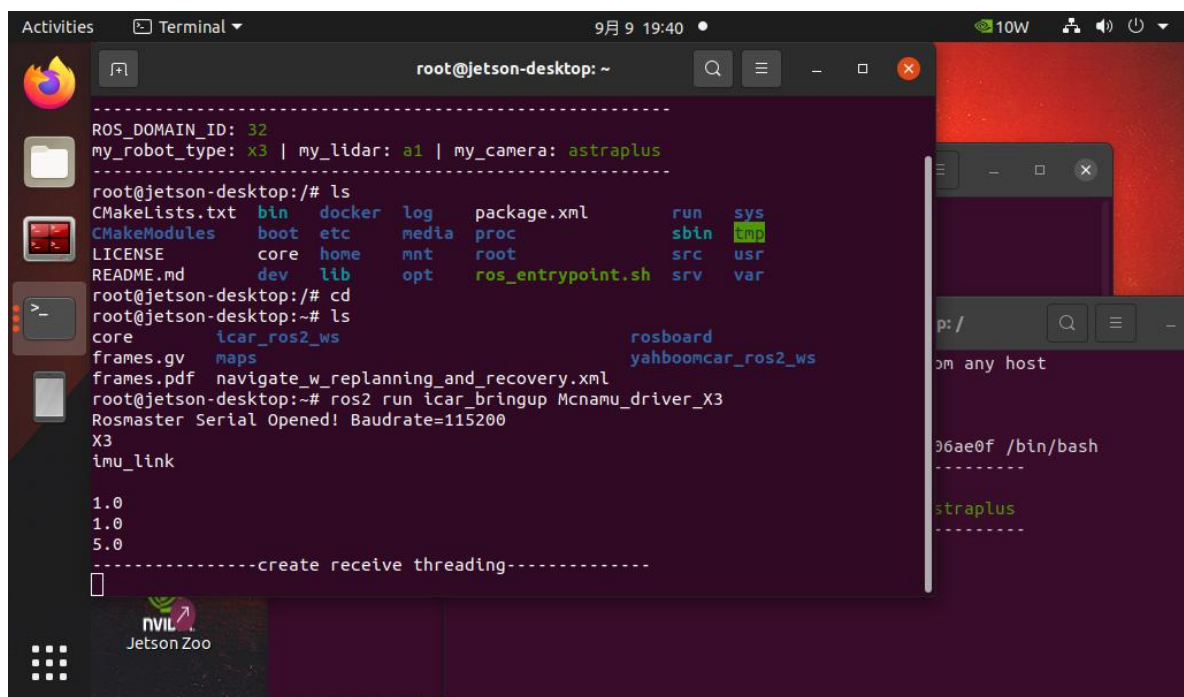
点来进行测试。

6.5 任务五 在 docker 镜像环境中运行预置的 ROS2 雷达避障功能

在上一个任务中，我们打开了三个 docker 环境 Terminal，按如下操作步骤，可以启动 iCar 智能小车系统中预置的 ROS2 雷达避障功能：

1. 在一个 docker 终端中启动 ROS2 底盘控制程序。

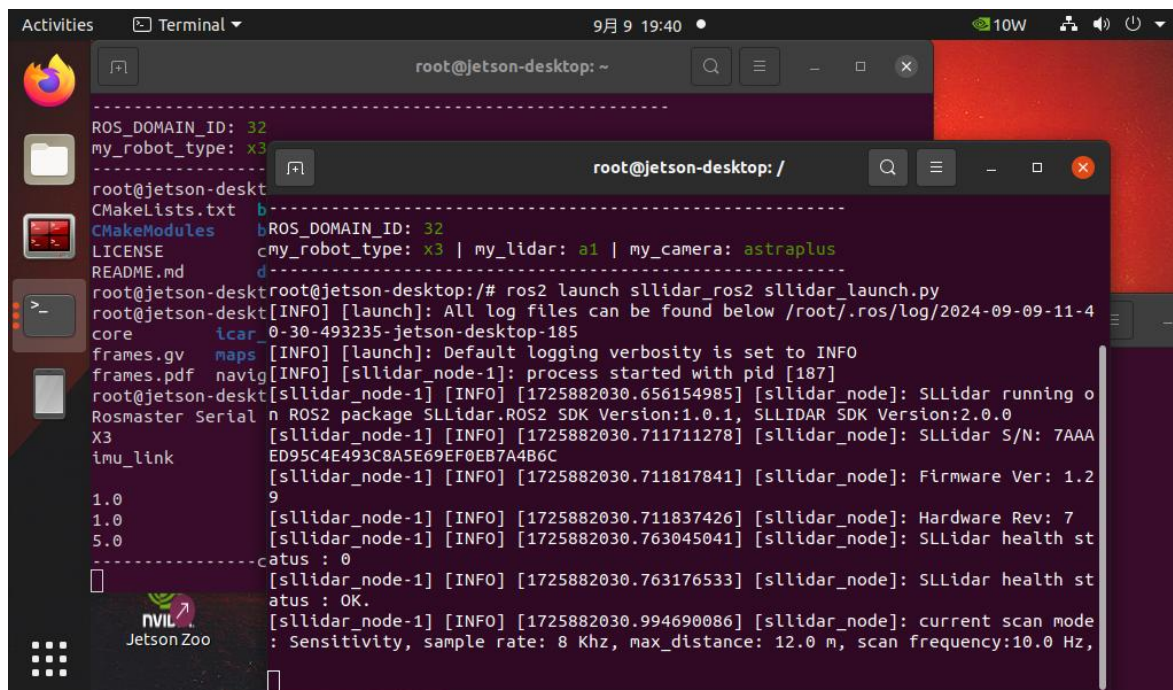
指令：ros2 run icar_bringup Mcnamu_driver_X3



```
root@jetson-desktop: ~  
-----  
ROS_DOMAIN_ID: 32  
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus  
-----  
root@jetson-desktop:~# ls  
CMakeLists.txt  bin  docker  log  package.xml  run  sys  
CMakeModules  boot  etc  media  proc  sbin  tmp  
LICENSE  core  home  mnt  root  src  usr  
README.md  dev  lib  opt  ros_entrypoint.sh  srv  var  
root@jetson-desktop:~# cd  
root@jetson-desktop:~# ls  
core  icar_ros2_ws  rosboard  
frames.gv  maps  yahboomcar_ros2_ws  
frames.pdf  navigate_w_replanning_and_recovery.xml  
root@jetson-desktop:~# ros2 run icar_bringup Mcnamu_driver_X3  
Rosmaster Serial Opened! Baudrate=115200  
X3  
imu_link  
  
1.0  
1.0  
5.0  
-----create receive threading-----  
[ ]
```

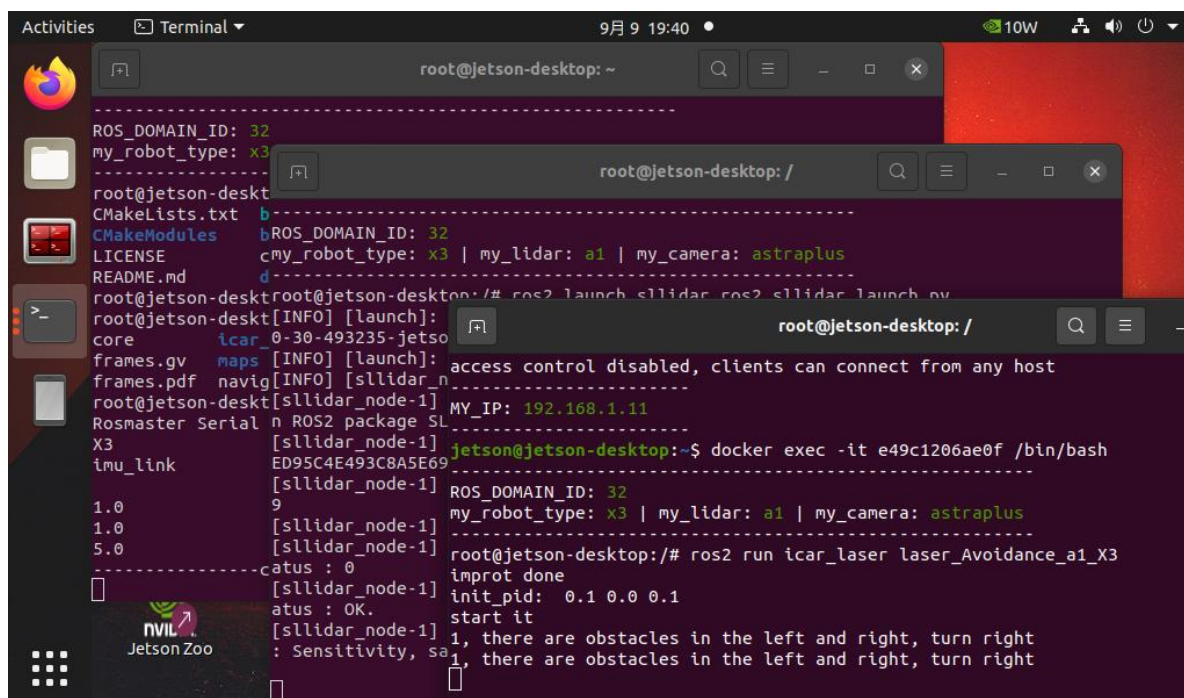
2. 在第二个 docker 终端中启动 ROS2 雷达扫描程序节点。

指令：ros2 launch slidar_ros2 slidar_launch.py



3. 在第三个 docker 终端中启动 ROS2 雷达避障应用程序节点。

指令: `ros2 run icar laser laser Avoidance a1 X3`



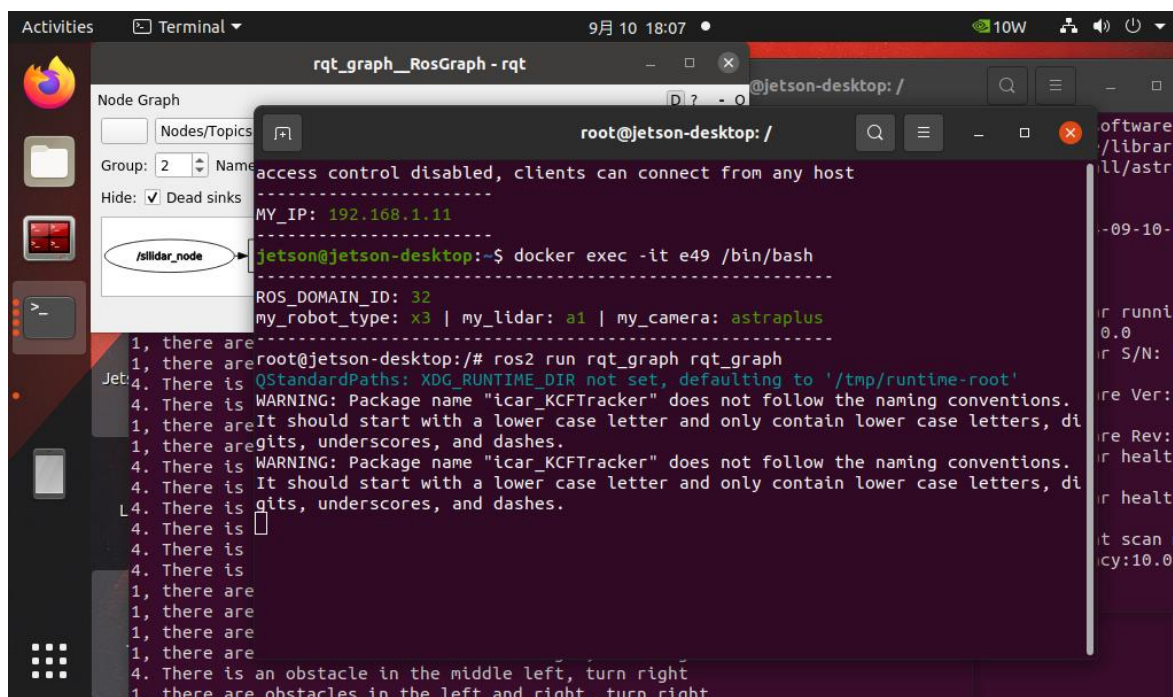
此时，拔掉 iCar 智能小车身上的键盘鼠标等连线，将 iCar 智能小车从四轮悬空的状态放到地面上，iCar 智能小车将开始自动前进，并在雷达扫描到前方有障碍物且障碍物接近

到一定距离时，进行自动避障运动：



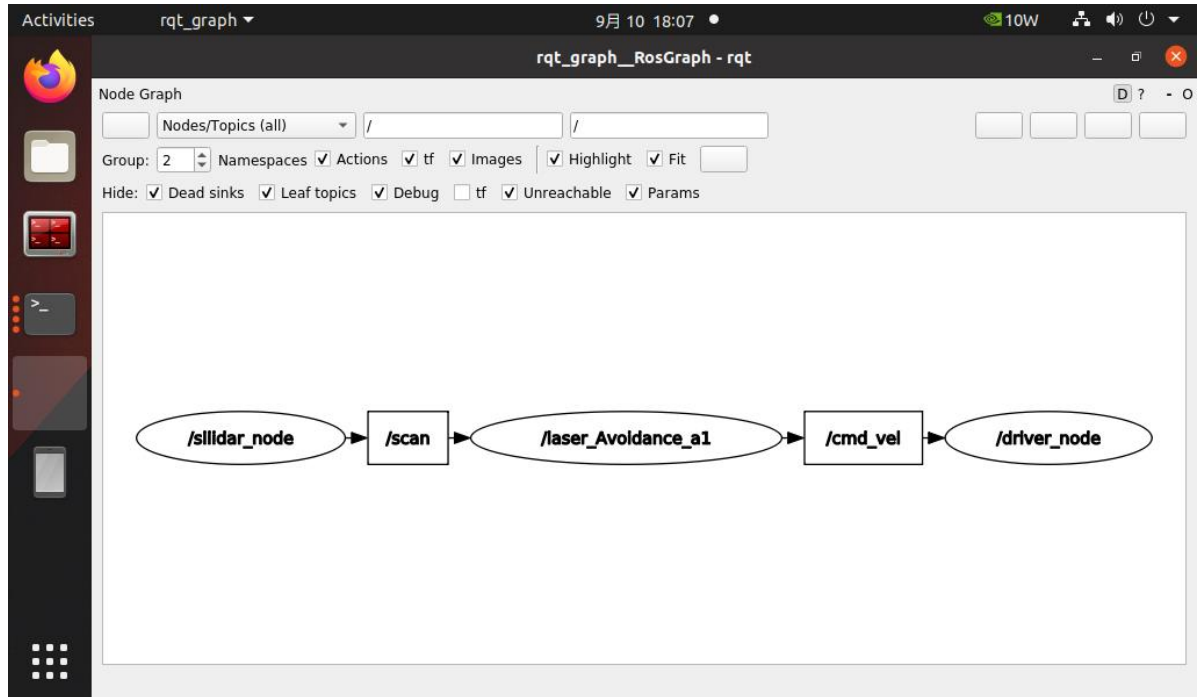
4. 观察 ROS2 中的话题消息关系

再开一个 docker 窗口，进入 ROS2 环境，运行指令 `ros2 run rqt_graph rqt_graph`



从 `rqt_graph` 的程序界面中，可以看到我们启动的三个 ROS2 应用以及它们之间的话题

关联：



首先，雷达扫描应用节点，将采集的雷达数据以 “/scan” 话题发布出来。

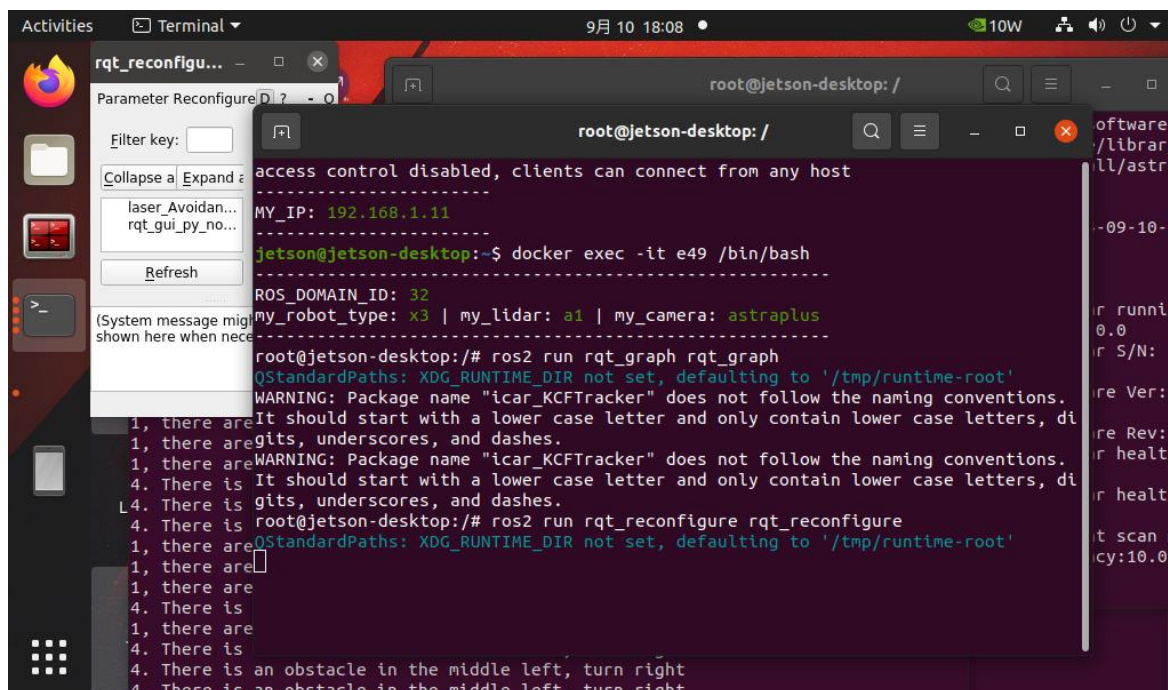
其次，雷达避障应用程序节点，订阅了 “/scan” 话题的数据，并对数据进行处理，在进行角度及距离的一系列判断后，转换成对底盘的运行速度和角度的控制话题 “/cmd_vel” 发布出来。

然后，底盘驱动应用程序，接收到速度控制话题 “/cmd_vel”，通过串口指令控制鸿蒙 AT 控制板驱动麦轮执行相应的运动动作。

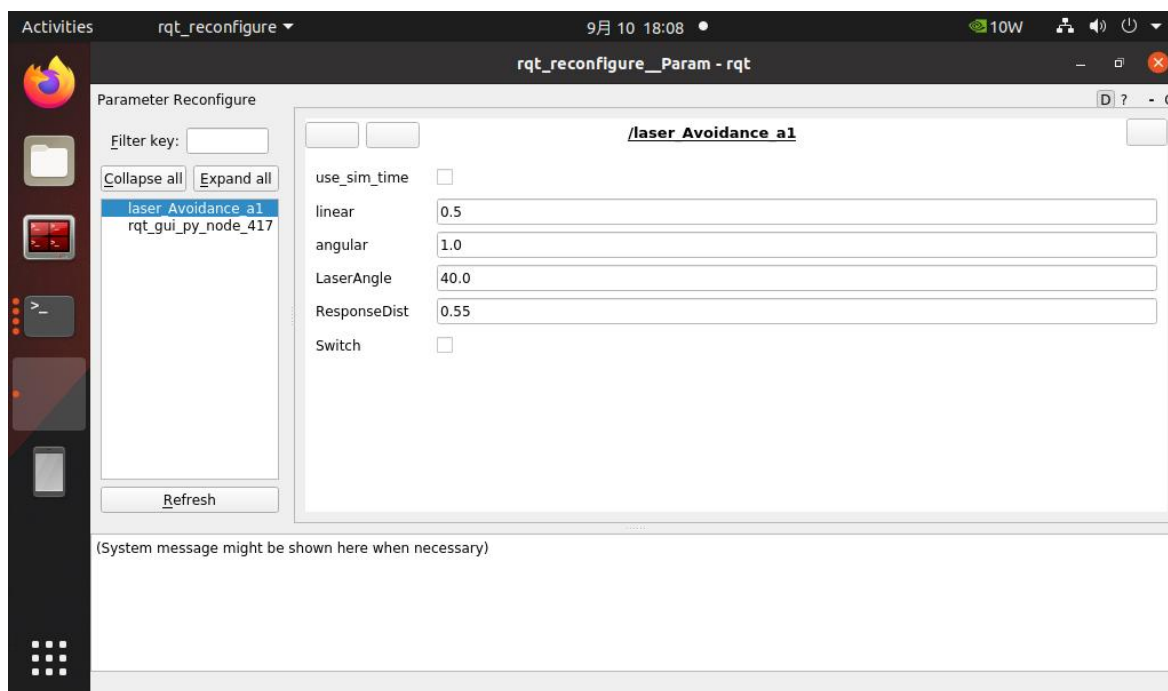
如此，不断循环，就能实现 iCar 智能小车在不断前进中，调整速度和角度，避开前方距离过近的障碍物。

5. 观察 ROS2 雷达避障应用的配置参数

再开一个 docker 窗口，进入 ros 环境，运行指令 `ros2 run rqt_reconfigure rqt_reconfigure`



在 rqt_reconfigure 程序界面中可以看到相关配置参数：



各参数含义如下，

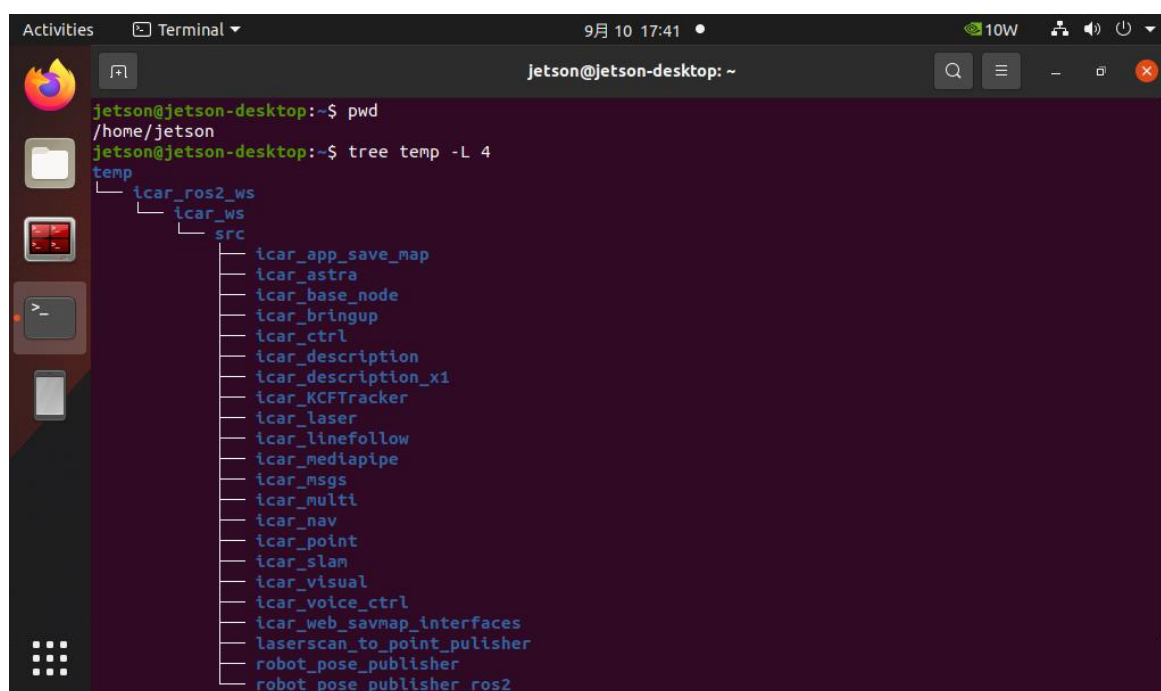
参数名称	参数含义
linear	线速度

angular	角速度
LaserAngle	雷达检测角度
ResponseDist	障碍物检测距离
Switch	玩法开关

以上参数都可调节，除了 Switch 以外，其他四个需要设置时候需要是小数，修改完，点击空白处才可以写入。修改不同的参数，观察 iCar 智能小车的雷达避障行为是否有变化。

6.6 任务六 修改 ROS2 雷达避障功能代码

1. 在 iCar 智能小车系统的如下路径中：~/temp，预置了相关功能代码：

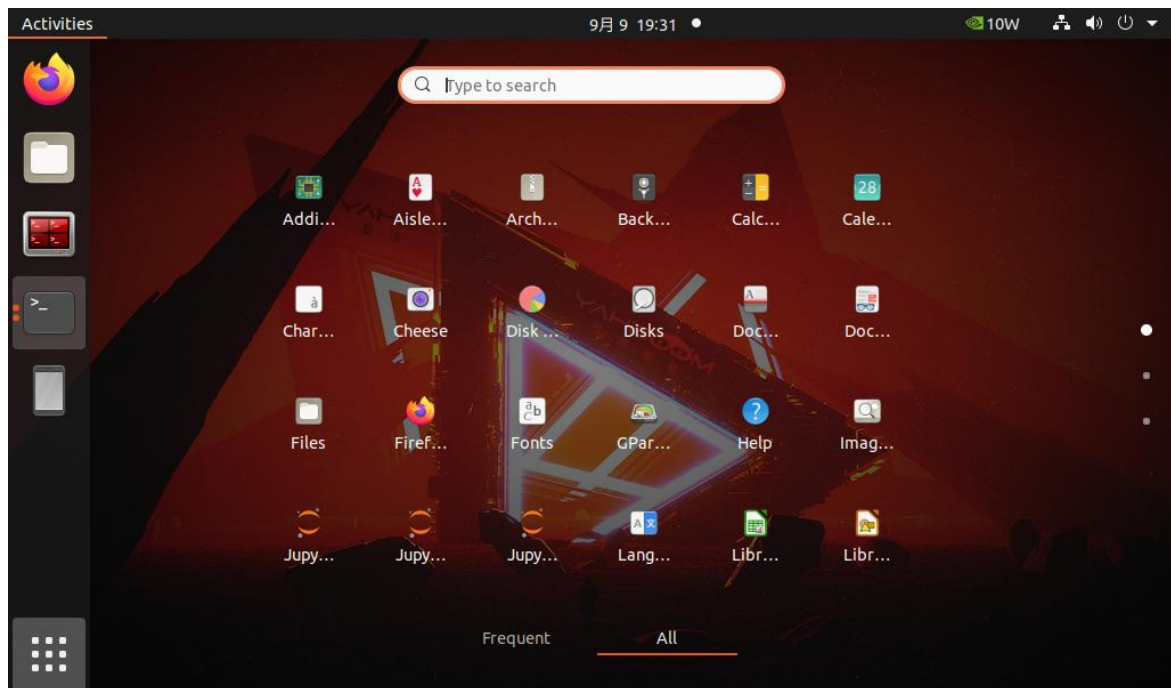


```

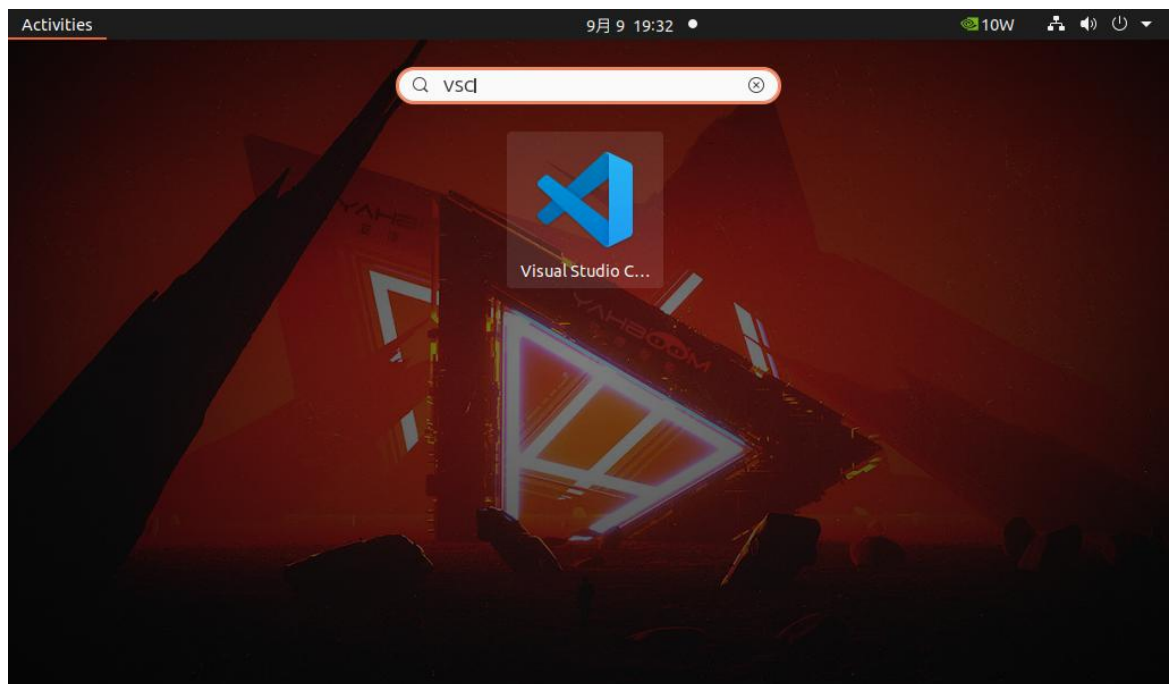
jetson@jetson-desktop:~$ pwd
/home/jetson
jetson@jetson-desktop:~$ tree temp -L 4
temp
├── icar_ros2_ws
│   └── icar_ws
│       └── src
│           ├── icar_app_save_map
│           ├── icar_astra
│           ├── icar_base_node
│           ├── icar_bringup
│           ├── icar_ctrl
│           ├── icar_description
│           ├── icar_description_x1
│           ├── icar_KCFTracker
│           ├── icar_laser
│           ├── icar_linefollow
│           ├── icar_mediapipe
│           ├── icar_msgs
│           ├── icar_multi
│           ├── icar_nav
│           ├── icar_point
│           ├── icar_slam
│           ├── icar_visual
│           ├── icar_voice_ctrl
│           ├── icar_web_savmap_interfaces
│           ├── laserscan_to_point_publisher
│           ├── robot_pose_publisher
│           └── robot_pose_publisher_ros2

```

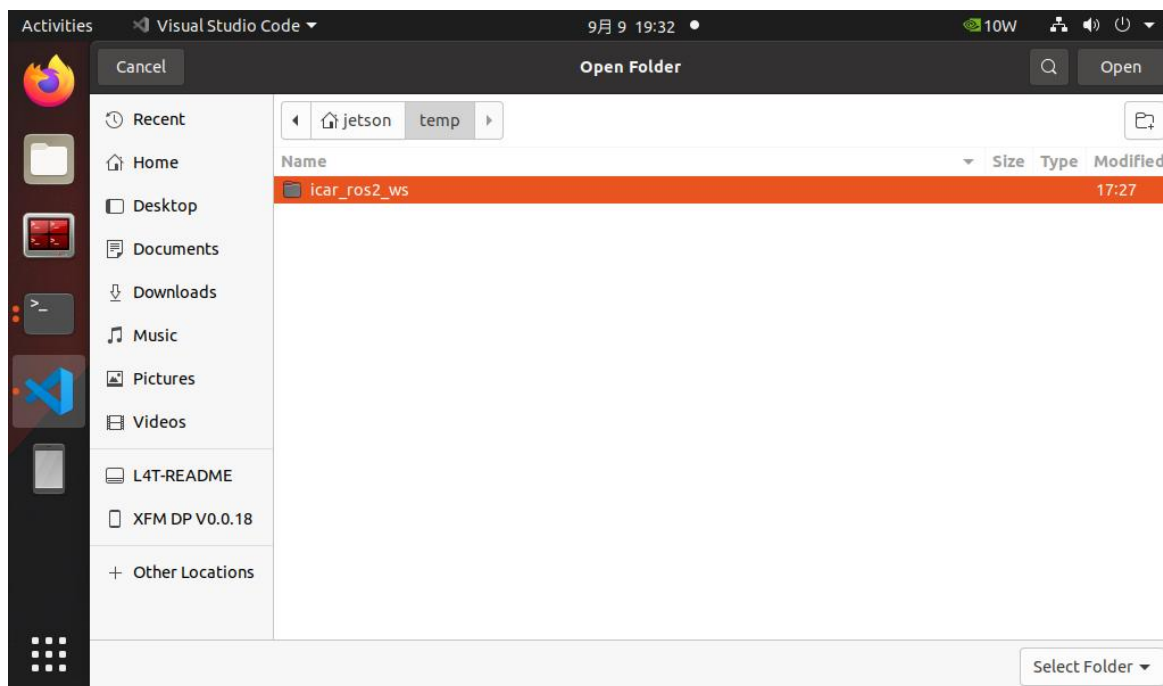
2. 点击左边栏的九宫格图标：



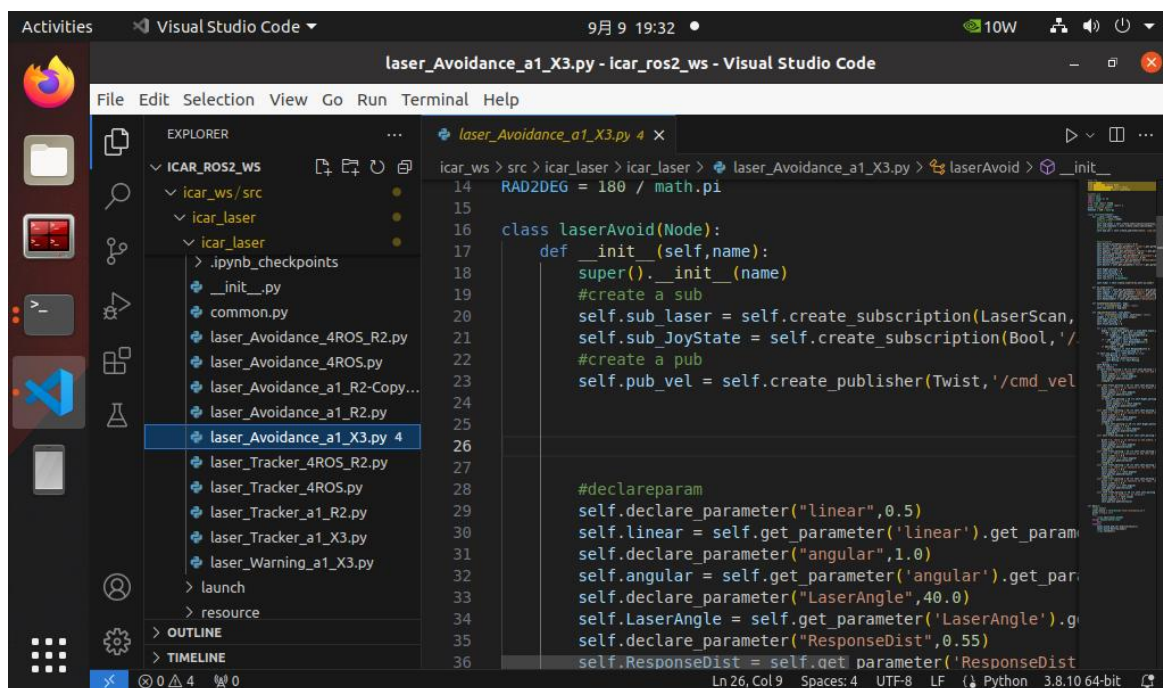
3. 搜索并启动已安装好的 VisualStudioCode 软件：

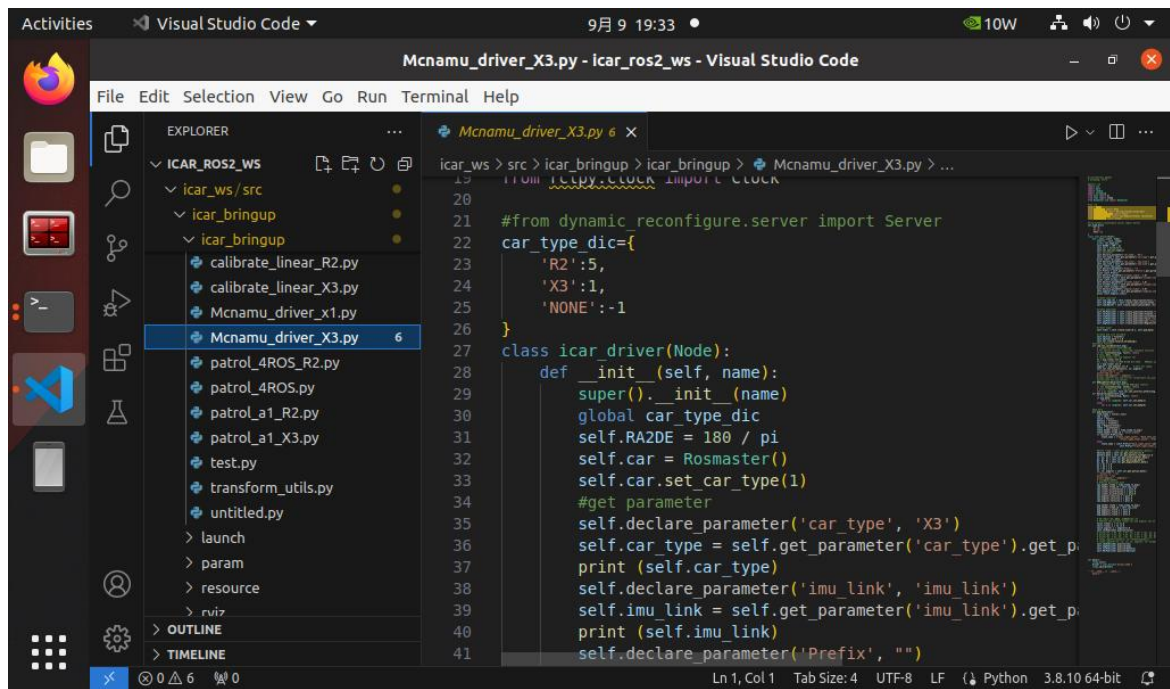


4. 使用 VSCode 打开步骤 1 中的代码：



5. 可以查看雷达避障或者底盘控制等 ROS2 相关代码：





雷达避障应用的初始化代码：

```
#ros lib
import rospy
from rospy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan

#common lib
import math
import numpy as np
import time
from time import sleep
from icar_laser.common import *
print ("improt done")
RAD2DEG = 180 / math.pi

class laserAvoid(Node):
    def __init__(self,name):
        super().__init__(name)
        #create a sub 订阅雷达数据话题，从雷达模块获取
        self.sub_laser = rospy.Subscriber('/scan', LaserScan, self.callback)
        self.create_subscription(LaserScan, "/scan", self.registerScan, 1)
        #create a pub 发布速度和角度的话题，向底盘模块发布
        self.pub_vel = self.create_publisher(Twist, '/cmd_vel', 1)
```



```
#declareparam 系统配置参数
self.declare_parameter("linear",0.5)
self.linear = self.get_parameter('linear').get_parameter_value().double_value
self.declare_parameter("angular",1.0)
self.angular = self.get_parameter('angular').get_parameter_value().double_value
self.declare_parameter("LaserAngle",40.0)
self.LaserAngle = self.get_parameter('LaserAngle').get_parameter_value().double_value
self.declare_parameter("ResponseDist",0.55)
self.ResponseDist = self.get_parameter('ResponseDist').get_parameter_value().double_value
self.declare_parameter("Switch",False)
self.Switch = self.get_parameter('Switch').get_parameter_value().bool_value

#中间变量，用于障碍物判断的逻辑控制
self.Right_warning = 0
self.Left_warning = 0
self.front_warning = 0
self.ros_ctrl = SinglePID()

#定时器，周期性检测配置参数更新
self.timer = self.create_timer(0.01,self.on_timer)

def on_timer(self):
    self.Switch = self.get_parameter('Switch').get_parameter_value().bool_value
    self.angular = self.get_parameter('angular').get_parameter_value().double_value
    self.linear = self.get_parameter('linear').get_parameter_value().double_value
    self.LaserAngle = self.get_parameter('LaserAngle').get_parameter_value().double_value
    self.ResponseDist = self.get_parameter('ResponseDist').get_parameter_value().double_value
```

对雷达测距数据的判断处理逻辑：

```
def registerScan(self, scan_data):
    if not isinstance(scan_data, LaserScan): return
    ranges = np.array(scan_data.ranges)
```

```

self.Right_warning = 0
self.Left_warning = 0
self.front_warning = 0

for i in range(len(ranges)):
    angle = (scan_data.angle_min + scan_data.angle_increment * i) * RAD2DEG
    if 160 > angle > 180 - self.LaserAngle:
        if ranges[i] < self.ResponseDist*1.5:
            self.Right_warning += 1
    if -160 < angle < self.LaserAngle - 180:
        if ranges[i] < self.ResponseDist*1.5:
            self.Left_warning += 1
    if abs(angle) > 160:
        if ranges[i] <= self.ResponseDist*1.5:
            self.front_warning += 1
self.Moving = True
twist = Twist()
    if self.front_warning > 10 and self.Left_warning > 10 and
self.Right_warning > 10:
        print ('1, there are obstacles in the left and right, turn right')
        twist.linear.x = self.linear
        twist.angular.z = -self.angular
        self.pub_vel.publish(twist)
        sleep(0.2)
        elif self.front_warning > 10 and self.Left_warning <= 10 and
self.Right_warning > 10:
        print ('2, there is an obstacle in the middle right, turn left')
        twist.linear.x = 0.0
        twist.angular.z = self.angular
        self.pub_vel.publish(twist)
        sleep(0.2)
        if self.Left_warning > 10 and self.Right_warning <= 10:
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.5)
            elif self.front_warning > 10 and self.Left_warning > 10 and
self.Right_warning <= 10:
            print ('4. There is an obstacle in the middle left, turn right')
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.2)

```

```

        if self.Left_warning <= 10 and self.Right_warning > 10:
            twist.linear.x = 0.0
            twist.angular.z = self.angular
            self.pub_vel.publish(twist)
            sleep(0.5)
        elif self.front_warning > 10 and self.Left_warning < 10 and
self.Right_warning < 10:
            print ('6, there is an obstacle in the middle, turn left')
            twist.linear.x = 0.0
            twist.angular.z = self.angular
            self.pub_vel.publish(twist)
            sleep(0.2)
        elif self.front_warning < 10 and self.Left_warning > 10 and
self.Right_warning > 10:
            print ('7. There are obstacles on the left and right, turn right')
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.4)
        elif self.front_warning < 10 and self.Left_warning > 10 and
self.Right_warning <= 10:
            print ('8, there is an obstacle on the left, turn right')
            twist.linear.x = 0.0
            twist.angular.z = -self.angular
            self.pub_vel.publish(twist)
            sleep(0.2)
        elif self.front_warning < 10 and self.Left_warning <= 10 and
self.Right_warning > 10:
            print ('9, there is an obstacle on the right, turn left')
            twist.linear.x = 0.0
            twist.angular.z = self.angular
            self.pub_vel.publish(twist)
            sleep(0.2)
        elif self.front_warning <= 10 and self.Left_warning <= 10 and
self.Right_warning <= 10:
            print ('10, no obstacles, go forward')
            twist.linear.x = self.linear
            twist.angular.z = 0.0
            self.pub_vel.publish(twist)

```

启动避障节点:

```

def main():
    rclpy.init()

```

```
laser_avoid = laserAvoid("laser_Avoidance_a1")

print("start it")

try:
    rclpy.spin(laser_avoid)
except KeyboardInterrupt:
    pass
finally:
    laser_avoid.pub_vel.publish(Twist())
    laser_avoid.destroy_node()
    rclpy.shutdown()
```

可以对上述代码逻辑进行修改或者优化, 比如修改对障碍物远近的距离判断的逻辑, 修改或优化后的代码需上传至 docker 中的 ROS2 代码工程中进行替换, 两者的对应关系如下:

VSCode 中的代码，在宿主机如下目录：

```

Activities 10W 9月 10 17:42
root@jetson-desktop: ~/icar_ros2_ws/icar_ws/src
fectionate_leakey
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
Error response from daemon: Container e49c1206ae0f7d19051bed4745e10616015c56b4378dfdcdba815982cbb548c is not running
jetson@jetson-desktop:~$ docker start e49
e49
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop:/# pwd
/
root@jetson-desktop:/# ls
CMakeLists.txt  README.md  core  etc  log  opt  root  sbin  sys  var
CMakeModules  bin  dev  home  media  package.xml  ros_entrypoint.sh  src  tcpp
LICENSE  boot  docker  lib  mnt  proc  run  srv  usr
root@jetson-desktop:/# src
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src# pwd
/root/icar_ros2_ws/icar_ws/src
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src# ls
build  icar_description  icar_nav  laserscan_to_point_publisher
icar_KCFTracker  icar_description_x1  icar_point  log
icar_app_save_map  icar_laser  icar_slam  robot_pose_publisher
icar_astra  icar_linefollow  icar_visual  robot_pose_publisher_ros2
icar_base_node  icar_mediapipe  icar_voice_ctrl
icar_bringup  icar_msgs  icar_web_savmap_interfaces
icar_ctrl  icar_multi  install
root@jetson-desktop:~/icar_ros2_ws/icar_ws/src#

```

docker 中的 ROS2 代码路径如下:

```

Activities  Terminal  9月 10 17:43  10W
root@jetson-desktop: ~/icar_ros2_ws/icar_ws/src

fectionate_leakey
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
Error response from daemon: Container e49c1206ae0f7d19051bed4745e10616015c56b4378fdcfdba4815982cbb548c is not running
jetson@jetson-desktop:~$ docker start e49
e49
jetson@jetson-desktop:~$ docker exec -it e49 /bin/bash
-----
ROS_DOMAIN_ID: 32
my_robot_type: x3 | my_lidar: a1 | my_camera: astraplus
-----
root@jetson-desktop:/# pwd
/
root@jetson-desktop:/# ls
CMakeLists.txt  README.md  core  etc  log  opt  package.xml  root  sbin  sys  var
CMakeModules   bin        dev  home media proc  ros_entrypoint.sh  src  tmp
LICENSE        boot      docker lib  mnt  run  run

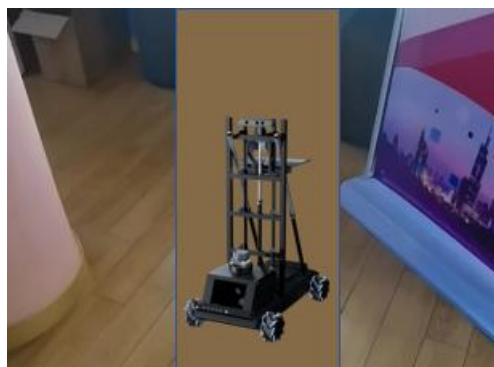
root@jetson-desktop:~/.ros2_ws/icar_ws/src# pwd
/root/icar_ros2_ws/icar_ws/src
root@jetson-desktop:~/.ros2_ws/icar_ws/src# ls
build          icar_description  icar_nav          laserscan_to_point_publisher
icar_KCFTracker  icar_description_x1  icar_point        log
icar_app_save_map  icar_laser          icar_slam         robot_pose_publisher
icar_astra        icar_linefollow    icar_visual       robot_pose_publisher_ros2
icar_base_node    icar_mediapipe     icar_voice_ctrl
icar_bringup      icar_msgs          icar_web_savmap_interfaces
icar_ctrl         icar_multi         install
root@jetson-desktop:~/.ros2_ws/icar_ws/src#

```

将代码上传到 docker 中替换以后，需在 docker 中的~/icar_ros2_ws/icar_ws 目录中执行指令：colcon build 来重新编译生成新的雷达避障应用节点。

然后，按任务五中的步骤，运行雷达避障功能，观察小车运动效果是否符合预期。

此时，拔掉键盘鼠标，将小车从四轮悬空的状态放到地面上，小车将开始自动前进，并在雷达扫描到有障碍物时，进行自动避障：

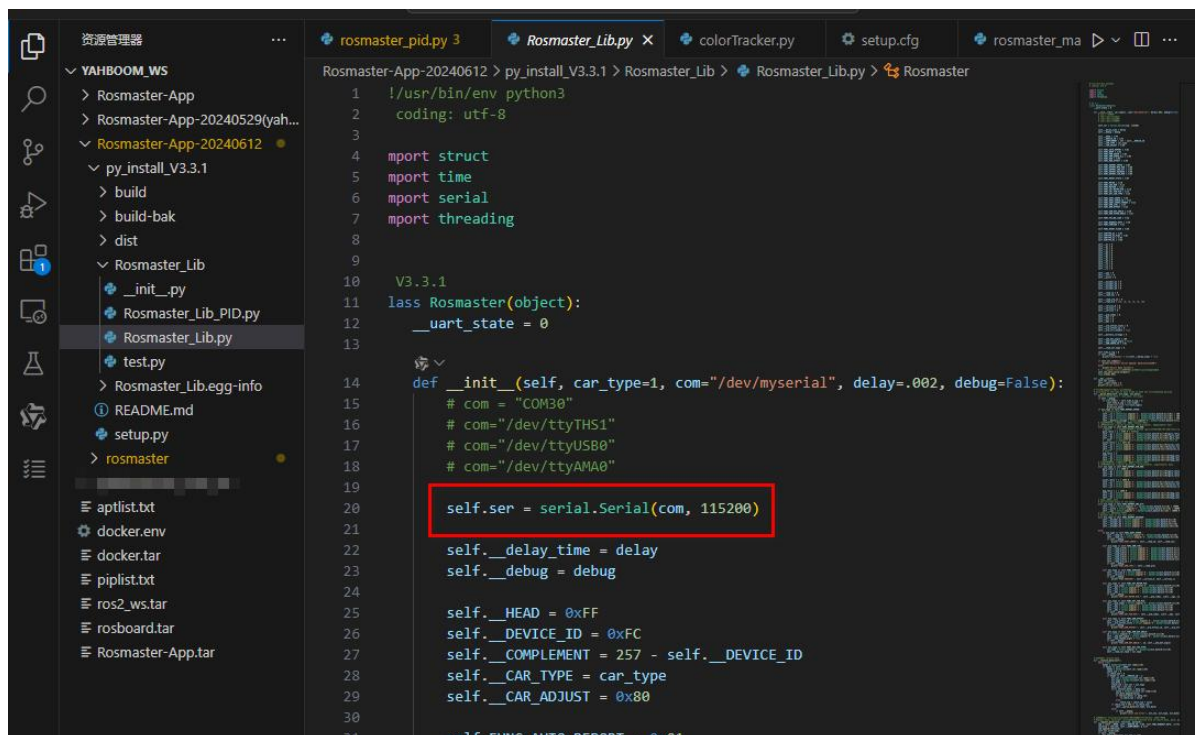


6.7 任务七 编写 python 代码控制底盘运动

在 iCar 智能小车的 Ubuntu 系统中，使用 pip3 list 命令可以看到 Rosmaster-Lib 库已经安装好。


```
jetson@jetson-desktop: ~
jetson@jetson-desktop: ~ 80x24
pyzbar 0.1.9
pyzmq 25.0.2
qtconsole 5.4.3
QtPy 2.3.1
requests 2.30.0
requests-oauthlib 2.0.0
requests-unixsocket 0.2.0
rfc3339-validator 0.1.4
rfc3986-validator 0.1.1
Rosmaster-Lib 3.3.1
rsa 4.9
scipy 1.5.3
seaborn 0.13.2
SecretStorage 2.3.1
Send2Trash 1.8.2
setuptools 65.5.1
simplejson 3.16.0
six 1.14.0
smbus2 0.4.3
smap 5.0.1
sniffio 1.3.0
soupsieve 2.4.1
spidev 3.6
stack-data 0.6.2
```

在这个库中，通过串口，可以和下位机底盘的基于 AT32 主板和 OpenHarmony 操作系统的控制系统进行通信，来控制底盘的运动：

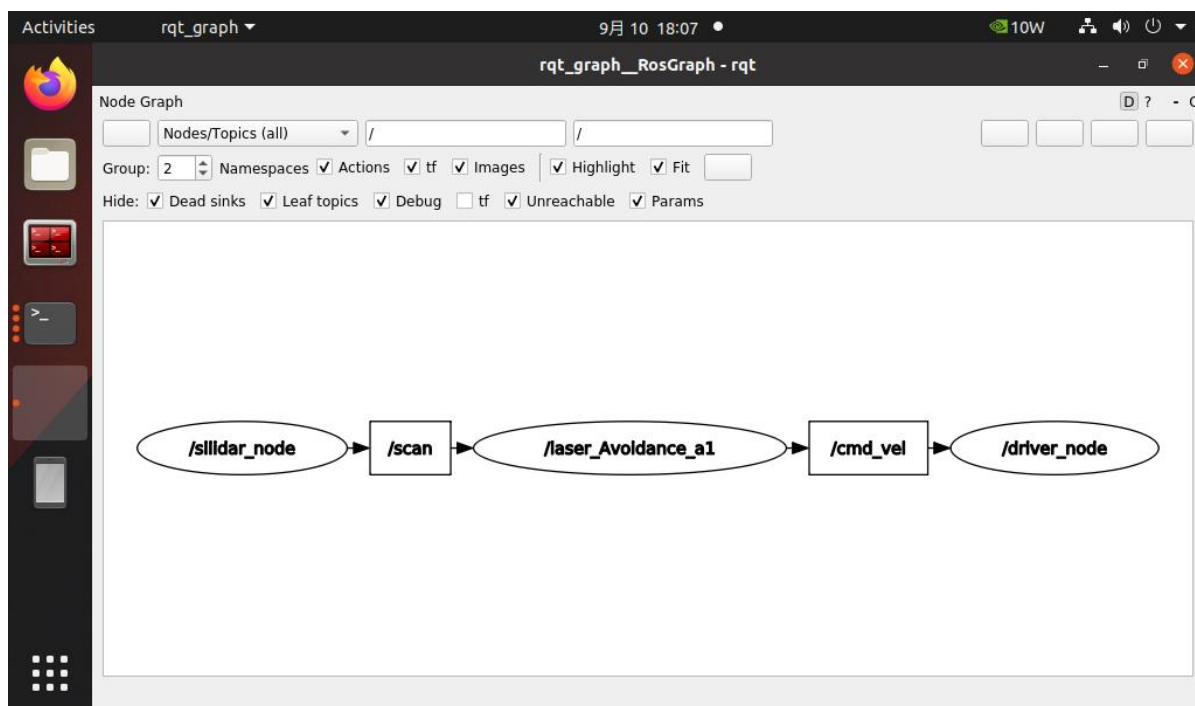


```

1  !/usr/bin/env python3
2  coding: utf-8
3
4  import struct
5  import time
6  import serial
7  import threading
8
9
10 V3.3.1
11 class Rosmaster(object):
12     __uart_state = 0
13
14     def __init__(self, car_type=1, com="/dev/myserial", delay=.002, debug=False):
15         # com = "COM30"
16         # com = "/dev/ttyTHS1"
17         # com = "/dev/ttyUSB0"
18         # com = "/dev/ttyAMA0"
19
20         self.ser = serial.Serial(com, 115200)
21
22         self.__delay_time = delay
23         self.__debug = debug
24
25         self.__HEAD = 0xFF
26         self.__DEVICE_ID = 0xFC
27         self.__COMPLEMENT = 257 - self.__DEVICE_ID
28         self.__CAR_TYPE = car_type
29         self.__CAR_ADJUST = 0x80
30
31         self.__FUNC_AUTO_REPORT = 0x01

```

使用 rqt_graph 工具查看话题关系：



在底盘节点中，也是在接收到运动调节话题/cmd_vel 后，调用 RosmasterLib 库的相关接口， 让小车底盘按指令执行运动：

```

src > icar_bringup > icar_bringup > Mcnamu_driver_x1.py > ...
7  import random
8  import threading
9  from math import pi
10 from time import sleep
11 from RosmasterLib import Rosmaster
12
13 #ros lib
14 import rclpy
15 from rclpy.node import Node
16 from std_msgs.msg import String,Float32,Int32,Bool
17 from geometry_msgs.msg import Twist
18 from sensor_msgs.msg import Imu,MagneticField, JointState
19 from rclpy.clock import Clock
20
21 #from dynamic_reconfigure.server import Server
22 car_type_dic={
23     'R2':5,
24     'X1':4,
25     'X3':1,
26     'NONE':-1
27 }
28 class icar_driver(Node):
29     def __init__(self, name):...
30     #callback function
31     def cmd_vel_callback(self,msg):
32         # 小车运动控制, 订阅者回调函数
33         # Car motion control, subscriber callback function
34         if not isinstance(msg, Twist): return
35         # 下发线速度和角速度
36         # Issue linear vel and angular vel
37         vx = msg.linear.x
38         #vy = msg.linear.y/1000.0*180.0/3.1416 #Radian system
39         vy = msg.linear.y
40         angular = msg.angular.z # wait for change
41         # self.get_logger().info("vx = {}, vy = {}, angular= {}".format(vx,vy,angul
42         self.car.set_car_motion(vx, vy, angular)
43         #rospy.loginfo("nav_use_rot:{}".format(self.nav_use_rotvel))
44         #print(self.nav_use_rotvel)
45     def RGBLightcallback(self,msg):...
46     def Buzzercallback(self,msg):...
47     #pub data
48     def pub_data(self):...
49
50     def main():
51         rclpy.init()
52         driver = icar_driver('driver_node')
53         rclpy.spin(driver)
54
55     '''if __name__ == '__main__':
56         main()'''

```

上述代码中的 cmd_vel_callback 就是在收到/cmd_vel 话题数据时,调用 RosmasterLib 的 set_car_motion 接口,控制底盘的运行速度。

我们可以在 iCar 智能小车的操作系统上编写如下代码 rosmaster_test.py, 来直接测试底盘运动控制的相关功能:

```

#!/usr/bin/env python3
# coding=utf-8
import os

```

```
import time
import sys
from Rosmaster_Lib import Rosmaster

g_debug = True
g_cmd = 1

if len(sys.argv) > 2:
    g_cmd = int(sys.argv[1])
    g_speed = int(sys.argv[2])

print(f'cmd:{g_cmd}')
print(f'speed:{g_speed}')

# 小车底层处理库
g_bot = Rosmaster(debug=g_debug)
# 启动线程接收串口数据
g_bot.create_receive_threading()

if __name__ == '__main__':

    # 控制小车向前、向后、向左、向右等运动。
    # state=[0, 7], =0 停止, =1 前进, =2 后退, =3 向左, =4 向右, =5 左旋, =6 右
    旋, =7 停车
    # speed=[-100, 100], =0 停止。
    # adjust=True 开启陀螺仪辅助运动方向。=False 则不开启。(此功能未开通)
    # Control the car forward, backward, left, right and other
    movements.
    # State =[0~6], =0 stop, =1 forward, =2 backward, =3 left, =4 right, =5
    spin left, =6 spin right
    # Speed =[-100, 100], =0 Stop.
    # Adjust =True Activate the gyroscope auxiliary motion
    direction. If =False, the function is disabled.(This function is not
    enabled)
    g_bot.set_car_run(g_cmd, g_speed, adjust=False)
    if (g_cmd == 7):
        os._exit(-1)

    # 主线程保持运行，直到用户中断
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
```

```
print("Exiting...")
```

先将 iCar 智能小车放到一个支撑物上，使得底盘的四个轮子悬空。

执行程序：python3 rosmaster_test.py 1 100

可以看见底盘的四个轮子以前进的方向转动。

执行程序：python3 rosmaster_test.py 7 100

可以看见底盘的四个轮子停止了转动。

同样可以使用不同的参数来测试底盘以不同的速度前进、后退、左移、右移、左转、右转、停止等。

#前进

python3 rosmaster_test.py 1 100

#后退

python3 rosmaster_test.py 2 100

#左平移

python3 rosmaster_test.py 3 100

#右平移

python3 rosmaster_test.py 4 100

#左转

python3 rosmaster_test.py 5 100

#右转

python3 rosmaster_test.py 6 100

#停止

python3 rosmaster_test.py 7 100

在 ROS2 的底盘控制程序节点中，也是通过类似上述调用，在 ROS2 应用节点中来控制底盘的运行。

7

实验结果

1. 学会了如何启动 iCar 智能小车及通过 APP 操控 iCar 智能小车的底盘运动。
2. 学会了如何进入 ROS2 的 docker 开发环境。
3. 学会了在 ROS2 的 docker 环境中启动运行雷达避障应用节点。
4. 学会了代码上传到 ROS2 的 docker 环境中并重新编译运行。
5. 学会了如何用代码控制 iCar 智能小车的运动。
6. 学会了如何开发部署雷达 ROS2 应用，获得并使用雷达测距扫描数据。
7. 学会了如何编写、修改雷达避障应用代码。

8

实验资源释放

1. 在各个 docker 运行终端中，按下 ctrl + c 键，退出程序运行。
2. 关闭各个 docker 终端。
3. 使用指令 `docker ps -a` 查看 docker 运行实例的 id。
4. 使用指令 `docker stop id` 来停止 docker 实例的运行。
5. 使用指令 `docker rm id` 来清除 docker 运行实例。
6. 关闭宿主机系统上的程序和终端窗口。

7. 在宿主机上启动一个终端窗口，输入指令 shutdown now，关闭系统。

9

实验小结

通过雷达避障功能的实验及相关代码开发，让开发者了解和掌握了 iCar 智能小车的启动及 APP 控制、docker 镜像操作、ROS2 开发环境中的应用节点启动及测试、雷达避障的应用开发、ROS2 中的话题交互机制等。为开发者在 iCar 智能小车平台上开发更多智能化应用打下了基础。